

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – ANALYTICAL PROBLEM SOLVING

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO2 – Manipulation (BL1, BL2, BL3)	Assessment:	Assessment Tool 1 – Analytical Problem Solving
Weightage:	40% of CIA	Total Marks:	100
CO2 Statement:	Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems (BL1, BL2, BL3)		
Student Name:	J KARTHIK	Reg. No.:	192472136
Section / Batch:	CSE – AI / 2024	Date:	

Instructions to Students

- Answer ALL questions. Total Marks: 100.
- Analyze each problem carefully before applying the appropriate Brute Force technique.
- Clearly show all intermediate steps, comparisons, iterations, and logical reasoning used in solving the problems.
- Apply suitable algorithms for sorting, searching, Closest-Pair, and Convex-Hull problems wherever required.
- Justify the correctness and efficiency of the proposed solution using appropriate complexity analysis.
- Use diagrams, tables, or stepwise illustrations wherever necessary for better explanation.
- Maintain neat presentation, proper algorithmic notation, and structured analytical reasoning throughout the answers.

Question Type	Focus Area	Questions	Marks
Application-Based Questions	Apply Brute Force techniques for sorting, searching, Closest-Pair and Convex-Hull problem analysis	Q1 Q2,Q3	40 60 (2X30)
GRAND TOTAL:		100 Marks	

SECTION A – APPLICATION BASED QUESTIONS

Code of Conduct

I J KARTHIK / 192472136 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: J KARTHIK_____

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20%	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30%	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20%	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20%	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10%	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Marks Evaluation:

Question No.	Understanding of Problem Context (20%)	Application of Concepts (30%)	Problem-Solving Approach (20%)	Accuracy of Solution (20%)	Clarity (10%)	Total Marks
Q1 (40 Marks)						
Q2 (30 Marks)						
Q3 (30 Marks)						
Overall Total (100)						

Faculty In-charge	Course Coordinator	Head of Department
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Question 1: Retail Inventory Price Sorting (Brute Force Sorting)

Application

A retail inventory management system needs to sort a small, unsorted list of product prices in ascending order before generating a price-range report. Since the dataset is small and no ordering assumption can be made, a Brute Force sorting technique — Selection Sort — is applied, which examines the entire unsorted portion of the list at every step to find the minimum element.

Brute Force Technique Used

Selection Sort

Selection Sort repeatedly scans the unsorted portion of the array to find the smallest (or largest) element and swaps it into its correct sorted position. This is a classic Brute Force strategy because it exhaustively compares every remaining element in each pass without any shortcuts.

Sample Input

Product prices (₹): [64, 25, 12, 22, 11]

Step-by-Step Solution

Pass	Array State	Minimum Found	Action
Initial	64, 25, 12, 22, 11	—	—
Pass 1	11, 25, 12, 22, 64	11 (index 4)	Swap with index 0
Pass 2	11, 12, 25, 22, 64	12 (index 2)	Swap with index 1
Pass 3	11, 12, 22, 25, 64	22 (index 3)	Swap with index 2
Pass 4	11, 12, 22, 25, 64	25 (index 3)	Already in place

Sorted Output: [11, 12, 22, 25, 64]

Complexity Analysis

Comparisons	$n(n-1)/2$ in all cases
Time Complexity	$O(n^2)$ — best, average, and worst case
Space Complexity	$O(1)$ — in-place, only a constant number of auxiliary variables

Justification: Selection Sort always performs the same number of comparisons regardless of the initial order of elements, since it must scan the full unsorted portion every pass. This makes it a true Brute Force method — correct but not adaptive to already-sorted or partially-sorted data.

Question 2: Library Book ISBN Lookup (Brute Force Searching)

Application

A library management system stores book ISBNs in an unsorted list (as books are added in the order they arrive, not sorted by ISBN). To check whether a specific ISBN exists before issuing a book, the system must search the list without assuming any order — the natural Brute Force approach is Linear Search.

Brute Force Technique Used

Linear (Sequential) Search

Linear Search checks every element of the list one by one, from the first to the last, comparing it against the search key until a match is found or the list is exhausted. It requires no preprocessing and works on unsorted data — the defining trait of a Brute Force search.

Sample Input

ISBN list: [118, 452, 204, 671, 305, 890] | Search key: 671

Step-by-Step Solution

Step	Index	Element	Comparison (== 671?)	Result
1	0	118	118 ≠ 671	Continue
2	1	452	452 ≠ 671	Continue
3	2	204	204 ≠ 671	Continue
4	3	671	671 = 671	Match Found

Result: ISBN 671 found at index 3 in 4 comparisons.

Complexity Analysis

Best Case	O(1) — key found at the first position
Worst Case	O(n) — key at the last position or absent
Average Case	O(n)
Space Complexity	O(1) — no extra memory required

Justification: Since the ISBN list is unsorted, no divide-and-conquer or indexed search (such as Binary Search) can be applied. Linear Search's exhaustive, element-by-element comparison is the correct and simplest Brute Force strategy for this scenario.

Question 3: Ride-Sharing Nearest Pickup Points (Closest-Pair Problem)

Application

A ride-sharing platform has multiple pending pickup requests plotted as coordinate points on a city map. To optimally cluster or merge nearby requests into a single trip, the system must identify the two closest pickup points among all requests — a classic Closest-Pair problem, solved here using the Brute Force approach.

Brute Force Technique Used

Brute Force Closest-Pair Algorithm

The Brute Force method computes the Euclidean distance between every possible pair of points and keeps track of the minimum distance found, without any spatial partitioning or sorting optimization.

Sample Input

Pickup points: A(2,3), B(12,30), C(40,50), D(5,1), E(12,10), F(3,4)

Step-by-Step Solution

Formula: $\text{distance}(P_i, P_j) = \sqrt{[(x_j - x_i)^2 + (y_j - y_i)^2]}$

All C(6,2) = 15 pairs are evaluated; the key comparisons are shown below:

Pair	Coordinates	Distance (approx.)
------	-------------	--------------------

Pair	Coordinates	Distance (approx.)
A – F	(2,3), (3,4)	1.41
A – D	(2,3), (5,1)	3.61
D – F	(5,1), (3,4)	3.61
A – E	(2,3), (12,10)	12.21
B – C	(12,30), (40,50)	34.53
... (remaining pairs)	—	larger values

Minimum Distance: pair A(2,3) and F(3,4) with distance ≈ 1.41

Complexity Analysis

Total Pairs Checked	$n(n-1)/2 = C(n,2)$
Time Complexity	$O(n^2)$
Space Complexity	$O(1)$ extra (excluding input storage)

Justification: The Brute Force Closest-Pair algorithm guarantees correctness by exhaustively checking every pair, making it reliable for small-to-moderate request volumes. For very large numbers of simultaneous ride requests, the $O(n^2)$ cost becomes expensive, motivating the divide-and-conquer Closest-Pair algorithm ($O(n \log n)$) as a future optimization.

Question 4: Delivery Zone Boundary Mapping (Convex-Hull Problem)

Application

A logistics company plots all customer delivery locations in a region as coordinate points and wants to compute the smallest convex boundary (delivery zone) that encloses all of them, to plan a perimeter patrol route. This is the classic Convex-Hull problem, solved here using the Brute Force method.

Brute Force Technique Used

Brute Force Convex-Hull Algorithm

For every pair of points (P_i, P_j), the algorithm draws the line through them and checks whether all remaining points lie entirely on one side of that line. If so, the segment P_iP_j is an edge of the convex hull.

Sample Input

Delivery points: $P_1(0,0)$, $P_2(4,0)$, $P_3(4,4)$, $P_4(0,4)$, $P_5(2,2)$

Step-by-Step Solution

Candidate Edge	Test: Do all other points lie on one side?	Hull Edge?
P1–P2	P3, P4, P5 all above the line	Yes
P2–P3	P1, P4, P5 all to the left	Yes
P3–P4	P1, P2, P5 all below the line	Yes
P4–P1	P2, P3, P5 all to the right	Yes
P1–P5, P2–P5, etc.	Other points fall on both sides	No (P5 is interior)

Convex Hull: $P_1(0,0) \rightarrow P_2(4,0) \rightarrow P_3(4,4) \rightarrow P_4(0,4) \rightarrow$ back to P_1 (P_5 lies inside and is excluded)

Complexity Analysis

Pairs of points tested	$O(n^2)$
Check per pair (all other points)	$O(n)$
Time Complexity	$O(n^3)$
Space Complexity	$O(n)$ to store hull points

Justification: The Brute Force Convex-Hull algorithm is correct and simple to implement but computationally expensive at $O(n^3)$, since it tests every pair of points against every other point. It is suitable for small delivery-point datasets; for larger regions, faster algorithms such as Graham's Scan ($O(n \log n)$) would be preferred.

Question 5: E-Commerce Product Rating Sort (Brute Force Sorting)

Application

An e-commerce platform wants to arrange a small batch of newly added product ratings in ascending order for a 'Top Rated' widget before more efficient indexing is applied. Given the small, unsorted batch, the Brute Force sorting technique — Bubble Sort — is used, which repeatedly compares and swaps adjacent elements.

Brute Force Technique Used

Bubble Sort

Bubble Sort repeatedly steps through the list, compares each pair of adjacent elements, and swaps them if they are in the wrong order. This process is repeated until no swaps are needed, exhaustively comparing all adjacent pairs each pass — a direct Brute Force strategy.

Sample Input

Product ratings: [3.8, 4.5, 2.9, 4.9, 3.2]

Step-by-Step Solution

Pass	Comparisons & Swaps	Array After Pass
Pass 1	3.8<4.5 (no swap); 4.5>2.9 (swap); 4.5<4.9 (no swap); 4.9>3.2 (swap)	3.8, 2.9, 4.5, 3.2, 4.9
Pass 2	3.8>2.9 (swap); 3.8<4.5 (no swap); 4.5>3.2 (swap)	2.9, 3.8, 3.2, 4.5, 4.9
Pass 3	3.8>3.2 (swap); 3.8<4.5 (no swap)	2.9, 3.2, 3.8, 4.5, 4.9
Pass 4	No swaps needed — array sorted	2.9, 3.2, 3.8, 4.5, 4.9

Sorted Output: [2.9, 3.2, 3.8, 4.5, 4.9]

Complexity Analysis

Best Case (already sorted)	$O(n)$ with early-exit optimization
Worst / Average Case	$O(n^2)$
Space Complexity	$O(1)$ — in-place sorting

Justification: Bubble Sort exhaustively compares every adjacent pair in every pass, which is characteristic of Brute Force sorting. It is simple and easy to verify for small rating batches, though it is not efficient for large-scale product catalogs, where $O(n \log n)$ sorts would be preferred.